Étude de cas : plan d'acquisition d'un satellite SPOT

Décider sous probabilités imprécises

Nahla BEN AMOR
nahla.benamor@gmx.fr

M2 Informatique – IA et Décision
Toulouse 2026-2027

D'après l'étude de cas de H. Fargier, M. Lemaître et G. Verfaillie

Plan

1. Le problème
2. Décider sous incertitude : nuages, pannes, critères
3. L'objectif de l'étude de cas

Plan

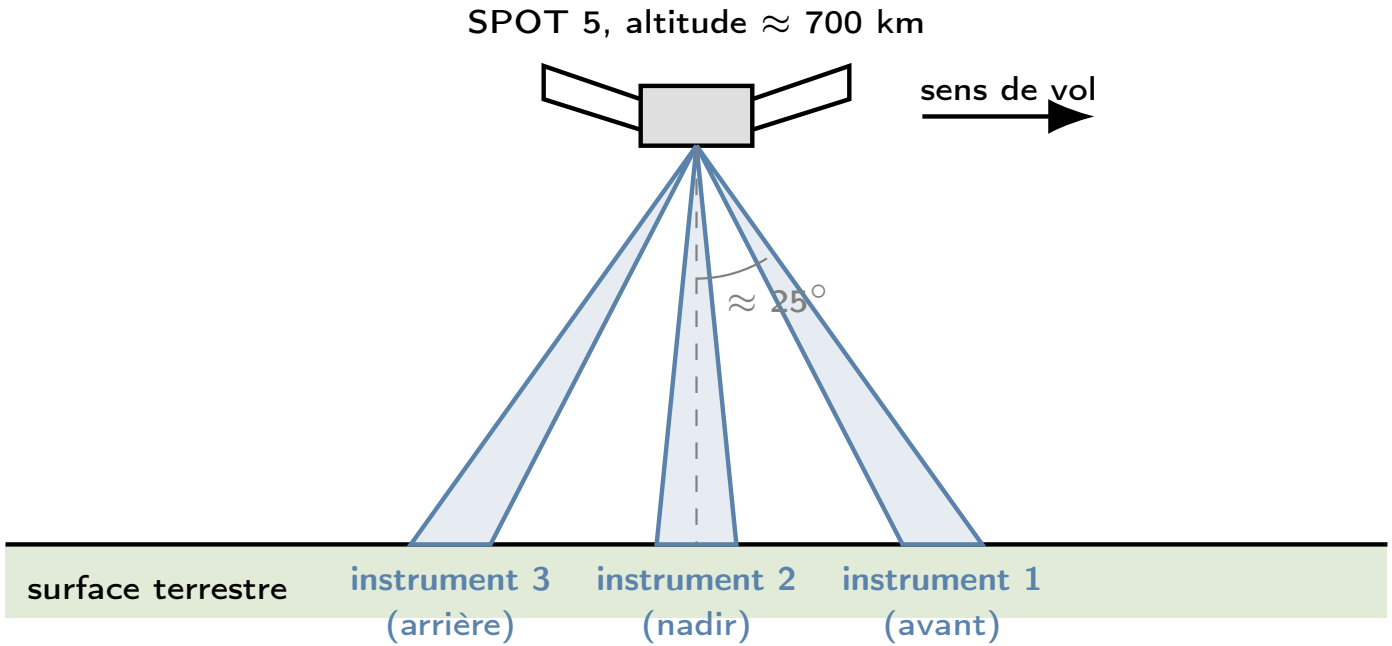
1 Le problème

2 Décider sous incertitude : nuages, pannes, critères

3 L'objectif de l'étude de cas

Introduction : SPOT, un satellite d'observation

SPOT – satellites optiques d'observation de la Terre (CNES / SPOT Image), de SPOT1 (1986) à SPOT7 (2014). Orbite héliosynchrone : un tour – une **révolution** – dure ≈100 min, soit ≈14 révolutions par jour, cycle de 26 jours pour revenir sur la même trace au sol. SPOT5 (notre étude) a **trois instruments** :

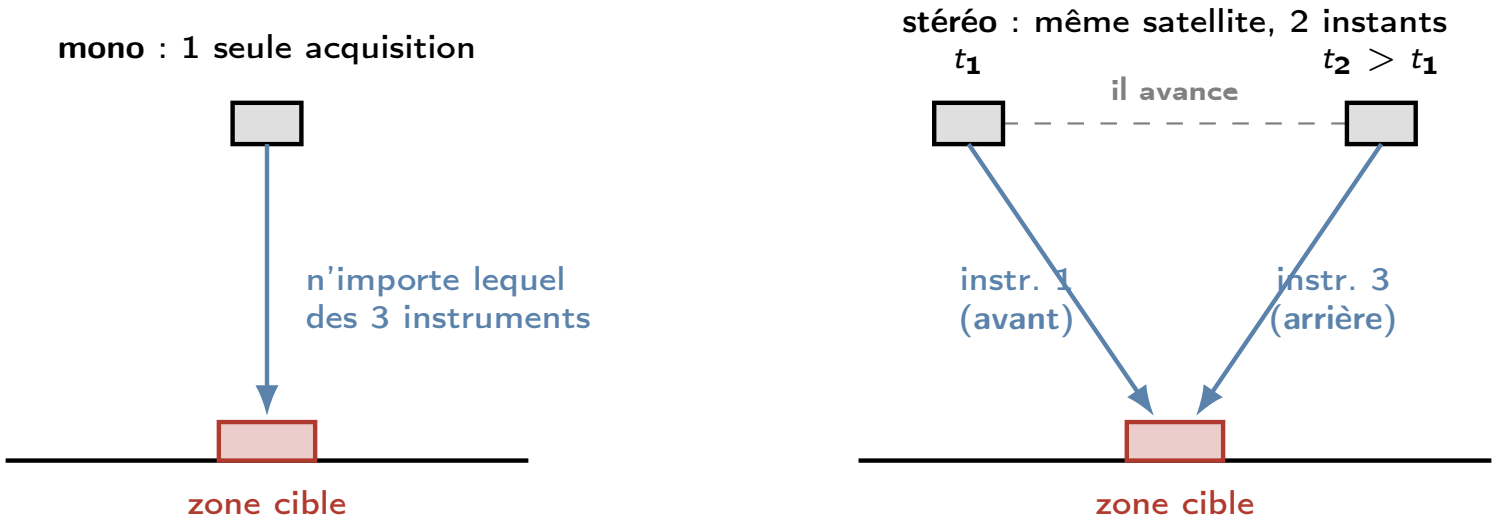


Miroir orientable par instrument : corridor de 800 km (400 km de chaque côté). Image transmise directement au sol si une station est à portée, sinon stockée à bord.

Le problème du jour

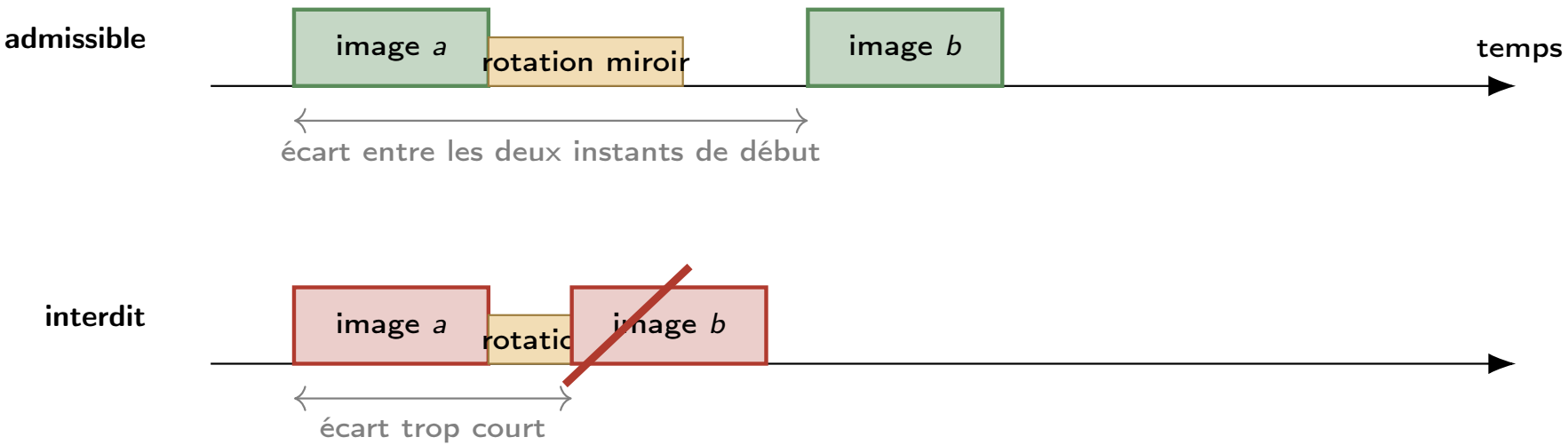
Chaque jour : décider **quelles images** seront prises le lendemain, et **comment** – gestion à court terme des commandes clients.

Image mono, image stéréo



La stéréo, c'est le même satellite qui survole la même zone et la photographie **deux fois**, sous deux angles : d'abord avec l'instrument **avant** (à t_1, quand la zone est encore devant lui), puis avec l'instrument **arrière** (à t_2, une fois qu'il l'a dépassée). Elle mobilise donc les instruments 1 et 3, jamais le nadir, et consomme **deux fois** la mémoire d'une mono. Conséquence pour la suite : si l'un des deux instruments tombe en panne, toute l'image est perdue.

Contrainte de chevauchement (1) : l'idée



Un instrument acquiert l'image *a* pendant une durée fixe, puis son miroir doit **pivoter** de l'angle de *a* vers l'angle de *b* avant de pouvoir commencer *b*. Si l'écart entre les deux dates de début ne couvre pas ces deux temps, les deux acquisitions se chevauchent : une seule des deux au plus peut être confiée à cet instrument.

Énoncé, contrainte 2. Elle est la seule des quatre à être déjà codée dans le programme fourni.

Exemple de commande : fichier spotProba1.py

```
nbImages = 3
TY = [1, 2, 1] # 1 = mono, 2 = stereo
PM = [10, 20, 10] # memoire (deja doublee pour la stereo)
PA = [10, 20, 40] # gain (prix de vente si selectionnee)

nbInstruments = 3 # toujours 3 : avant, nadir, arriere

# DD[i][j] = instant de debut d'acquisition de l'image i par l'instrument j (s)
DD = [[130, 230, 330], [150, 0, 350], [220, 320, 420]]

# AN[i][j] = angle de depointage du miroir pour l'image i, instrument j (deg)
AN = [[10, 10, 10], [5, 0, 5], [20, 20, 20]]

DU = 20 # duree d'acquisition, identique pour toutes les images (s)
VI = 1 # vitesse de rotation du miroir (deg/s)
PMmax = 50 # capacite memoire maximale

ProbaInf = [0, 0, 0] # borne inf de la proba de nuage
ProbaSup = [0, 0, 0] # borne sup de la proba de nuage
Failure = [0, 0, 0] # proba de panne de chaque instrument
```

Ce que cette lecture confirme

C'est une **commande de 3 images** (2 mono + 1 stéréo), **sans incertitude** (`ProbaInf` = `ProbaSup` = 0, `Failure` = 0) – gain optimal attendu : **70**. Le détail de `DD` et `AN` est expliqué sur la diapo suivante.

Contrainte de chevauchement (2) : les données

Nom	Taille	Signification	Unité	Exemple (jeu 1)

`DD[i][j]`	$n \times 3$	instant de début d'acquisition de l'image `i` par l'instrument `j`, compté depuis le début de la révolution	s	`DD[0][0] = 130`
				`DD[0][1] = 230`
				`DD[0][2] = 330`

`AN[i][j]`	$n \times 3$	angle de dépointage latéral du miroir pour viser l'image `i` depuis `j`	deg	`AN[0][0] = 10`
				`AN[0][1] = 10`
				`AN[0][2] = 10`

`DU`	scalaire	durée d'acquisition, la même pour toutes les images	s	`DU = 20`
`VI`	scalaire	vitesse de rotation du miroir	deg/s	`VI = 1`

Lire `DD[0] = [130, 230, 330]`

C'est la 1ère image (indice `0` en Python), **mono** (`TY[0]=1`) : **trois possibilités mutuellement exclusives** pour la même image – par l'avant à 130 s, par le nadir à 230 s, par l'arrière à 330 s.

Le satellite voit la zone d'abord devant lui, puis dessous, puis derrière : d'où les 100 s d'écart, temps de parcours des 25° entre deux instruments.

Lire `AN[0] = [10, 10, 10]`

La zone est à 10° **sur le côté** de la trace au sol, soit environ 123 km. Les trois instruments suivent la **même trace** : ils doivent donc tous pivoter du même angle. D'où trois valeurs identiques.

Pour l'image stéréo, `AN[1] = [5, 0, 5]` : 5° pour l'avant et l'arrière, et 0 en colonne nadir car ce créneau n'existe pas (comme `DD[1][1] = 0`).

Dans la contrainte, seul compte l'**écart** $|\texttt{AN[a][j]} - \texttt{AN[b][j]}|$, pas l'angle absolu.

Instruments 1, 2, 3 dans l'énoncé, indexés 0, 1, 2 en Python : la stéréo porte sur 0 et 2.

Premier survol du code : spotProbaPartial.py

Le programme fourni suit toujours le même squelette en cinq blocs.

1. **Import des données** : `from spotProba1 import ...` – ce fichier travaille toujours sur le jeu 1, en dur, quel que soit le jeu qu'on souhaite tester. À corriger : le programme final doit être générique, capable de charger n'importe quel jeu passé en paramètre.
2. **Variables de décision** : `selection[i]` (booléen : image `i` retenue ou non) et `assignedTo[i][j]` (booléen : image `i` affectée à l'instrument `j`).
3. **Fonction objectif** :

```
mymodel.setObjective(quicksum(PA[i] * selection[i]
                    for i in range(nbImages)), sense='maximize')
```

   `quicksum(... for i in range(nbImages))` joue le rôle de $\sum_i$; chaque terme vaut `PA[i]` si l'image est retenue (`selection[i]=1`), sinon 0. Au total : $\max \sum_i$ `PA[i]` $\cdot$ `selection[i]` – la somme des gains, sans incertitude (ni nuage, ni panne). À corriger.
4. **Résolution et affichage** : appel au solveur SCIP, puis boucle qui imprime les couples image/instrument retenus.

La seule contrainte codée : le non-chevauchement

```
for ima1, ima2 in product(range(nbImages), range(nbImages)):
    if ima1 < ima2:
        for ins in range(nbInstruments):
            if abs(DD[ima1][ins]-DD[ima2][ins])*VI < DU*VI \
               + abs(AN[ima1][ins]-AN[ima2][ins]):
                mymodel.addCons(
                    assignedTo[ima1][ins] + assignedTo[ima2][ins] <= 1)
```

Son fonctionnement est détaillé sur les diapos suivantes.

Contrainte de chevauchement (3) : la règle

L'idée, en une phrase

Un instrument doit **pivoter** son miroir avant de passer d'une image à l'autre. Si deux images sont programmées trop près l'une de l'autre sur le **même** instrument, ce temps de pivot n'a pas le temps de s'écouler : impossible de les confier toutes les deux à cet instrument.

La règle, pour deux images a, b et un instrument j

Si

$$\underbrace{|\texttt{DD[a][j]} - \texttt{DD[b][j]}|}_{\text{écart de temps}} \times \texttt{VI} < \underbrace{\texttt{DU} \times \texttt{VI}}_{\text{durée d'acquisition, en degrés}} + \underbrace{|\texttt{AN[a][j]} - \texttt{AN[b][j]}|}_{\text{écart d'angle}}$$

alors

$$\texttt{assignedTo[a][j]} + \texttt{assignedTo[b][j]} \leq 1$$

`VI` = vitesse de rotation du miroir (deg/s) : un temps multiplié par `VI` donne l'angle que le miroir parcourt pendant ce temps. Tout est ainsi comparé **en degrés**. Avec `VI=1` (jeu 1), un écart de 20 s équivaut à 20°.

Pourquoi ≤ 1 et pas ≤ 0 ou < 1 ?

Somme de deux booléens : 0, 1 ou 2. Seul **2** (les deux **en même temps**) est interdit – d'où « ≤ 1 », sans empêcher a ou b d'aller sur un **autre** instrument.

La règle appliquée : un exemple concret

Sur les images 0 et 1, instrument 0 (`spotProba1`)

	image 0	image 1
`DD[.][0]`	130 s	150 s
`AN[.][0]`	10°	5°

$$\underbrace{|150 - 130|}_{=20} < \underbrace{20 + |10 - 5|}_{=25} \quad \Rightarrow \text{conflit}$$

Conséquence

$20 < 25$: le miroir n'a pas le temps de pivoter entre les deux dates de début. On ajoute donc au modèle :

$$\texttt{assignedTo[0][0]} + \texttt{assignedTo[1][0]} \leq 1$$

À retenir

Le même `j` apparaît des deux côtés : on compare deux images **sur un instrument donné**. Les 3 miroirs sont indépendants, donc on répète ce test **séparément pour chaque instrument** – une paire d'images peut être en conflit sur l'un et parfaitement compatible sur un autre. C'est exactement ce qu'on va vérifier, systématiquement, sur la diapo suivante.

Tous les couples du jeu spotProba1

3 images, 3 instruments $\Rightarrow \binom{3}{2} \times 3 = 9$ tests. $DU = 20$ s, $VI = 1$ deg/s.

Paire	Instr.	Écart $\lvert\Delta DD\rvert$	Requis $DU + \lvert\Delta AN\rvert$	Résultat
0–1	0	20 s	25 s	conflit
0–1	1	230 s*	30 s*	ok
0–1	2	20 s	25 s	conflit
0–2	0	90 s	30 s	ok
0–2	1	90 s	30 s	ok
0–2	2	90 s	30 s	ok
1–2	0	70 s	35 s	ok
1–2	1	320 s*	40 s*	ok
1–2	2	70 s	35 s	ok

Pourquoi des paires, pas des triplets ? Sur une même ligne temporelle, si **chaque paire** est compatible, l'ensemble l'est aussi : un conflit à 3 impliquerait déjà un conflit dans l'une des paires testées.

*** Instrument 1 (nadir) pour l'image 1**

L'image 1 est **stéréo** : elle n'utilise jamais le nadir. Les valeurs `DD[1][1]=0` et `AN[1][1]=0` sont de simples **valeurs de remplissage**, pas de vraies dates. Le test tombe « ok » ici uniquement parce que les **vraies** dates des autres images sur cet instrument (230 s, 320 s) sont très loin de 0 – avec d'autres chiffres, ce test sur une valeur de remplissage aurait pu créer une contrainte absurde.

Conséquence sur le plan optimal

Deux contraintes seulement sont ajoutées (paire 0–1, instruments 0 et 2). L'image 1 (stéréo) occupe les instruments 0 et 2 : l'image 0, en conflit avec elle sur ces deux-là, n'a plus le choix que du nadir – le modèle ne le choisit pas par préférence, il y est contraint.

Les données d'une révolution (un tour du satellite, ≈100 min)

Donnée	Taille	Signification
`nbImages`	scalaire	nombre de commandes d'images
`nbInstruments`	scalaire	vaut 3
`TY[i]`	n	1 = mono, 2 = stéréo
`PA[i]`	n	prix de vente de l'image i
`PM[i]`	n	mémoire consommée (déjà doublée pour la stéréo)
`PMmax`	scalaire	capacité mémoire totale
`DD[i][j]`	$n \times 3$	date de début d'acquisition de i sur j (s)
`AN[i][j]`	$n \times 3$	angle de dépointage du miroir (degrés)
`DU`	scalaire	durée d'acquisition, commune
`VI`	scalaire	vitesse de rotation du miroir (deg/s)
`ProbaInf[i]`	n	borne inférieure de la proba de nuage sur i
`ProbaSup[i]`	n	borne supérieure de la proba de nuage sur i
`Failure[j]`	3	probabilité de panne de l'instrument j

Lecture des deux derniers tableaux

$p_i \in [0{,}7\,;\,0{,}9]$ se lit « environ 0,8 de risque de nuage, pas moins de 0,7 ni plus de 0,9 ».
$p_i \in [0\,;\,0]$: ciel certainement dégagé. $p_i \in [1\,;\,1]$: zone certainement couverte.

Les quatre contraintes de faisabilité

1. **Mémoire** : la somme des PM_i des images retenues ne dépasse pas $PMmax$.
2. **Non-chevauchement** : deux images sur le même instrument exigent un délai suffisant pour faire pivoter le miroir.
3. **Stéréo** : une image stéréo occupe les instruments 1 et 3, et seulement ceux-là.
4. **Exclusivité** : un instrument ne réalise pas deux acquisitions simultanées.

Ce qui est donné, ce qui reste à faire

Ces 4 contraintes sont énoncées ici en langage naturel, telles que le sujet les formule. Seule la contrainte 2 est **déjà traduite en équations et codée** (diapos précédentes). Traduire les contraintes 1 et 3 en équations, et les ajouter au modèle, fait partie du travail demandé.

La contrainte 4 est en fait impliquée par la contrainte 2 : deux acquisitions simultanées se chevauchent nécessairement.

Plan

1 Le problème

2 Décider sous incertitude : nuages, pannes, critères

3 L'objectif de l'étude de cas

Nuages : probabilité précise vs imprécise

Une taille par tableau, pas la même pour tous

Jeu fictif à 4 images, 3 instruments :

```
ProbaSup = [0, 0.1, 0.4, 0] # 4 valeurs : une par image
Failure = [0, 0.05, 0.9] # 3 valeurs : une par instrument
```

1. Nuage **précis**

```
ProbaInf = [0.1, 0.1, 0.1]
ProbaSup = [0.1, 0.1, 0.1]
```

`ProbaInf = ProbaSup` : valeur **connue avec certitude** (10 %). Chaque gain $\times$ 0,9. Objectif = 63 (au lieu de 70).

2. Nuage **imprécis**

```
ProbaInf = [0.1, 0.1, 0.1]
ProbaSup = [0.3, 0.3, 0.3]
```

`ProbaInf < ProbaSup` : risque **entre 10 % et 30 %**. Pire cas retenu (30 %). Objectif = 49.

Pannes d'instruments, et le cas combiné

3. Panne totale d'un instrument

```
ProbaInf = [0, 0, 0]
ProbaSup = [0, 0, 0]
Failure = [0, 1, 0]
```

Ciel garanti clair, mais le **nadir** (indice 1) est **certainement en panne** (`Failure[1]=1`). Toute image qui lui serait affectée rapporterait un gain nul. Objectif = 60 (on perd une possibilité d'affectation).

4. Tout combiné, variable par image/instrument

```
ProbaInf = [0, 0, 0]
ProbaSup = [0.1, 0.5, 0.9]
Failure = [0.01, 0.9, 0.01]
```

Chaque image a **son propre** risque de nuage, chaque instrument **sa propre** fiabilité (nadir très peu fiable : 90 %). Objectif = 14,661 – l'arbitrage devient difficile à deviner à l'œil, d'où le besoin d'un calcul systématique.

Pondérer le gain par le risque

L'idée

Une image ne vaut pas son prix plein si elle risque d'être nuageuse : retenue, elle compte pour $PA_i \times (1 - p_i)$, où p_i est la probabilité qu'elle soit ratée (nuage).

Exemple

Avec $PA_i = 40$ et $p_i = 0{,}70$:

$$PA_i \times (1 - p_i) \;=\; 40 \times 0{,}30 \;=\; 12$$

Mais quelle valeur de p_i ?

On n'a qu'un **intervalle** $[p_i^{\text{inf}}, p_i^{\text{sup}}]$, pas une probabilité connue avec certitude. Laquelle des deux bornes retenir – ou un compromis entre les deux – dépend de la **règle de décision** choisie (votre cours). C'est à vous de choisir et de justifier.

Plan

1 Le problème

2 Décider sous incertitude : nuages, pannes, critères

3 L'objectif de l'étude de cas

L'objectif, tel que formulé dans l'énoncé

Objectif de l'étude de cas

« Déterminer les **meilleurs plans d'acquisition** pour les problèmes décrits dans les fichiers `spotProba4` et `spotProba5`, qui impliquent respectivement **20 et 40 images**. »

- Dans `spotProba4`, les instruments sont supposés totalement fiables (probabilité de panne nulle).
- Dans `spotProba5`, il existe des probabilités de défaillance non négligeables, notamment pour l'instrument 3.

Moyens

« Vous construirez un modèle du problème basé sur la **programmation linéaire mixte** et/ou la **programmation par contraintes**. Un exemple d'un tel programme est fourni, qui implémente la plupart des contraintes. Ce programme est en Python et appelle le solveur SCIP via la bibliothèque `pyscipopt`. Vous pouvez utiliser cet environnement et compléter simplement le modèle. »

Jeux de test

Les jeux 1, 2 et 3 (3 images : 2 mono, 1 stéréo) servent à tester et déboguer. Sans incertitude, les gains valent respectivement 70, 60 et 60. Les variantes avec incertitude figurent **en commentaires** dans le premier fichier.

Questions ouvertes par l'énoncé

L'énoncé fixe le but (les deux plans) mais laisse plusieurs choix à votre charge. Ce sont eux qui feront la différence entre deux rendus.

Sur la modélisation

- Quel **critère** appliquer, et pourquoi celui-là ?
- Sous quelles **hypothèses** l'utilité espérée se décompose-t-elle image par image ?

Sur les résultats

- Dans `spotProba5`, l'instrument 3 est peu fiable, et toute image stéréo en dépend. Comment le plan se déforme-t-il par rapport à `spotProba4` ?
- Quel écart entre le plan pessimiste et le plan optimiste ? Les deux plans diffèrent-ils seulement par leur valeur, ou aussi par les **images retenues** ?
- Y a-t-il des images systématiquement écartées, et pour quelle raison : mémoire, chevauchement, ou trop de risque ?

Note pratique

L'énoncé mentionne des fichiers `.dat` ; les fichiers réellement distribués sont des modules Python `.py`, importés directement par le programme.